

Practice Contents

- Step 1: Creating Your First Shell Script

- a) Open a terminal window and log in to your Linux VM.
- b) Use a text editor to create a file named *script1* with the following contents:

```
#!/bin/bash

echo You are $USER

echo Your home directory is $HOME

echo Your home directory consists of `du -sh ~`
```

- c) Save the file and exit the text editor.
- d) Change the permissions of the script to 745 using the following command:

```
chmod 745 script1
```

- e) Run the script by typing the following command:
- f) What is the output of the script?

```
HE210528@OSG203:~$ ./script1
You are HE210528
Your home directory is /home/HE210528
Your home directory consists of 168504 /home/HE210528
HE210528@OSG203:~$
```



- g) Alter the script so that the output of each echo statement is redirected to the file *info.txt*.
- h) How did you do this?

Thêm > info.txt vào sau \$USER

Thêm >> info.txt vào sau \$HOME và `du -sh ~`



- Step 2: Getting User Input

- a) Use a text editor to create a file named *script2* with the following contents:

```
#!/bin/bash

echo What is your name?
```

```

read NAME
echo What is your username?
read USERNAME
echo Hello $NAME, your home directory contents and size:

```

- b) Complete the script by adding instructions to output the user's home directory contents, assuming that the user's home directory will be /home/\$USERNAME, as well as using the du statement from *script1* to output the disk usage of the user's home directory.
- c) Save the file and exit the text editor.
- d) Change the permissions of the script to 745.
- e) Run the script and answer the prompts.
- f) What is the output of the script?

```

HE210528@OSG203:~$ ./script2
What is your name?
khanh
What is your username?
HE210528
Hello khanh, your home directory contents and size:
168512 /home/HE210528
HE210528@OSG203:~$

```

- Step 3: Using Parameters

- a) Rewrite script2 so that the NAME and USERNAME are supplied to the script by parameters, and remove the first four instructions (the first two echo and the two read statements).

```

GNU nano 8.1 script2
#!/bin/bash
NAME=$1
USERNAME=$2

echo Hello $NAME, your home directory contents and size:
du -sH /home/$USERNAME

```

- b) Save the file and exit the text editor.
- c) Run the script with your name and username as parameters.
- d) What is the output of the script?

```

HE210528@OSG203:~$ ./script2 "khanh" HE210528
Hello khanh, your home directory contents and size:
168512 /home/HE210528
HE210528@OSG203:~$

```

- Step 4: Using Conditional Statements

- a) Use a text editor to create a file named *script4* with the following contents:

```

#!/bin/bash
if [ $# -ne 2 ]; then echo Illegal input
elif [ $1 -gt $2 ]; then echo $1 is greater
else echo $2 is greater
fi

```

- b) Save the file and exit the text editor.
- c) Change the permissions of the script to 745.
- d) Run the script, providing it with no parameters, two parameters of 5 and 10, two parameters of 10 and 5, and two parameters of 5 and 5.
- e) What does [\$# -ne 2] mean?
 Biểu thức \$# đại diện cho số lượng tham số truyền vào script. Toán tử -ne có nghĩa là "Not Equal" (Không bằng)
- f) Do we need this in the script?
 Có vì nó giúp kiểm tra tính hợp lệ của dữ liệu đầu vào, ngăn chặn lỗi logic xảy ra nếu người dùng quên không nhập đủ hoặc nhập thừa tham số khi chạy script
- g) Show how you would alter the script so that it can also output if the two parameters are the same.
 Thêm nhánh điều kiện elif [\$1 -eq \$2]

```
#!/bin/bash
if [ $# -ne 2 ]; then echo Illegal input
elif [ $1 -eq $2 ]; then echo Both parameters are the same
elif [ $1 -gt $2 ]; then echo $1 is greater
else echo $2 is greater
fi
```

- Step 5: Using Loops
 - a) Use a text editor to create a file named *script5* with the following contents:


```
#!/bin/bash
read -p "Enter the number you seek " NUM
for VALUE in $@; do
if [ $VALUE -eq $NUM ]; then COUNT=$((COUNT+1)); fi
done
echo $NUM appeared $COUNT times
```
 - b) Save the file and exit the text editor.
 - c) Change the permissions of the script to 745.
 - d) Run *script5*, passing it the list of numbers 5 10 6 12 5 18 10 4 19 21 5 12 18 22, and when prompted, input 5.


```
HE210528@OSG203:~$ ./script5 5 10 6 12 5 18 10 4 19 21 5 12 18 22
Enter the number you seek 5
5 appeared 3 times
HE210528@OSG203:~$
```

Xuất hiện 3 lần
 - e) Rerun the script, inputting 18 instead.


```
HE210528@OSG203:~$ ./script5 5 10 6 12 5 18 10 4 19 21 5 12 18 22
Enter the number you seek 18
18 appeared 2 times
```

Xuất hiện 2 lần
 - f) Rerun the script, inputting 23 instead.

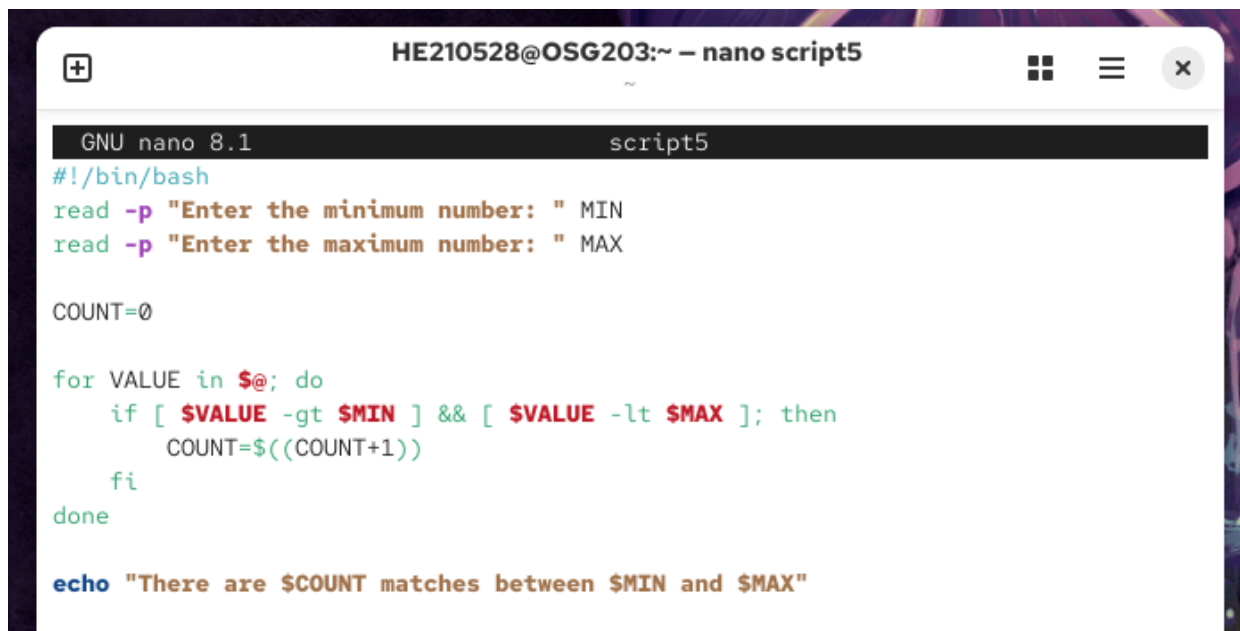
```
HE210528@OSG203:~$ ./script5 5 10 6 12 5 18 10 4 19 21 5 12 18 22
Enter the number you seek 23
23 appeared times _
```

Xuất hiện 0 lần

- g) What outputs did you get for each input?
- Với input là 5: Kết quả trả về 5 appeared 3 times.
 - Với input là 18: Kết quả trả về 18 appeared 2 times.
 - Với input là 23: Kết quả trả về 23 appeared times (không hiển thị gì nghĩa là 0)
- h) Rewrite *script5* to input two inputs instead of just NUM and count the number of parameters that fall between the two. For instance, if the input is 10 and 20, we would have 5 matches (12, 18, 19, 12, 18).

```
HE210528@OSG203:~$ ./script5 5 10 6 12 5 18 10 4 19 21 5 12 18 22
Enter the minimum number: 10
Enter the maximum number: 20
There are 5 matches between 10 and 20
HE210528@OSG203:~$
```

Rewrite của tôi:



```
HE210528@OSG203:~ - nano script5
GNU nano 8.1 script5
#!/bin/bash
read -p "Enter the minimum number: " MIN
read -p "Enter the maximum number: " MAX

COUNT=0

for VALUE in $@; do
    if [ $VALUE -gt $MIN ] && [ $VALUE -lt $MAX ]; then
        COUNT=$((COUNT+1))
    fi
done

echo "There are $COUNT matches between $MIN and $MAX"
```

- Step 6: Finding the Largest and Smallest
 - a) Write a script that receives a list of parameters and outputs the largest and smallest of the list.



```
HE210528@OSG203:~ - nano script6
GNU nano 8.1 script6 Modified
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Error: No parameters supplied."
    exit 1
fi
MAX=$1
MIN=$1
for VALUE in $@; do
    if [ $VALUE -gt $MAX ]; then
        MAX=$VALUE
    fi
    if [ $VALUE -lt $MIN ]; then
        MIN=$VALUE
    fi
done
echo "Largest: $MAX"
echo "Smallest: $MIN"
```

- b) If the user does not pass any parameters, output an error message.\

```
HE210528@OSG203:~$ ./script6
Error: No parameters supplied.
```

- c) Test out your script to make sure it works in a variety of cases (no parameters, 1 parameter, 5 parameters, etc).

```
HE210528@OSG203:~$ ./script6 1 2 3 4 5 6 7 8 9 10 50 99
Largest: 99
Smallest: 1
```

- Step 7: Checking File Permissions

- a) Use a text editor to create a file named *script8* with the following contents:

```
#!/bin/bash

for file in $@; do

if [[ -r $file && -w $file && -x $file ]]

then count=$((count+1))

fi

done

echo Of the $# files entered, $count were r/w/x
```

- b) Save the file and exit the text editor.
- c) Change the permissions of the script to 745.
- d) Run the script to make sure it works.
- e) Modify the script in three ways:
 - Output an error message if no parameters are supplied.
 - Rather than counting the number of files, add up the size of these files.
 - Output the average size of the file (which will be the sum/\$#).

```
HE210528@OSG203:~ -- nano script8
GNU nano 8.1 script8 Modified
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Error: No files specified as parameters."
    exit 1
fi
TOTAL_SIZE=0
MATCH_COUNT=0
for file in $@; do
    if [ [ -f $file && -r $file && -w $file && -x $file ] ]; then
        FILE_SIZE=$(stat -c%s "$file")
        TOTAL_SIZE=$((TOTAL_SIZE + FILE_SIZE))
        MATCH_COUNT=$((MATCH_COUNT + 1))
    fi
done
echo "Total size of r/w/x files: $TOTAL_SIZE bytes"
if [ $# -gt 0 ]; then
    AVERAGE_SIZE=$((TOTAL_SIZE / $#))
    echo "Average size relative to total inputs: $AVERAGE_SIZE bytes"
fi
```

- Step 8: Counting File Types
 - a) Write a script that receives a directory name and iterates through all of the items in this directory, counting the number of items that are regular files, that are directories, and that are links, and output this result along with the total number of items being reported on.

```
#!/bin/bash
```

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 <directory_path>"
    exit 1
fi
```

```
TARGET_DIR=$1
```

```
if [ ! -d "$TARGET_DIR" ]; then
    echo "Error: $TARGET_DIR is not a valid directory."
    exit 1
fi
```

```
REGULAR_FILES=0
DIRECTORIES=0
SYMLINKS=0
TOTAL_ITEMS=0
```

```
for item in "$TARGET_DIR"/* "$TARGET_DIR"/.*; do
```

```

if [[ "$(basename "$item")" == "." || "$(basename "$item")" == ".." ]]; then
    continue
fi

if [ -e "$item" ] || [ -L "$item" ]; then
    TOTAL_ITEMS=$((TOTAL_ITEMS + 1))

    if [ -L "$item" ]; then
        SYMLINKS=$((SYMLINKS + 1))
    elif [ -f "$item" ]; then
        REGULAR_FILES=$((REGULAR_FILES + 1))
    elif [ -d "$item" ]; then
        DIRECTORIES=$((DIRECTORIES + 1))
    fi
fi
done

echo "Directory Report for: $TARGET_DIR"
echo "-----"
echo "Regular Files: $REGULAR_FILES"
echo "Directories:  $DIRECTORIES"
echo "Symbolic Links:$SYMLINKS"
echo "-----"
echo "Total Items:  $TOTAL_ITEMS"

```

- b) Run your script on several directories like /etc, /bin, and ~.

/etc:

```

HE210528@OSG203:~$ ./script9 /etc
Directory Report for: /etc
-----
Regular Files: 100
Directories:   139
Symbolic Links:10
-----
Total Items:   249

```

/bin:

```
HE210528@OSG203:~$ ./script9 /bin
```

```
Directory Report for: /bin
```

```
-----  
Regular Files: 1391
```

```
Directories: 0
```

```
Symbolic Links:294
```

```
-----  
Total Items: 1685
```

```
~:
```

```
HE210528@OSG203:~$ ./script9 ~
```

```
Directory Report for: /home/HE210528
```

```
-----  
Regular Files: 13
```

```
Directories: 15
```

```
Symbolic Links:0
```

```
-----  
Total Items: 28
```